Discrete Mathematics

Yifan Li

August 2026

Contents

Introduction **5**

1 Logic and Proofs **6**
1.1 Propositions and Logical Operators 7
1.2 Logical Equivalences 8
1.3 Predicate Logic 8
1.4 Quantifiers 9
1.5 Negation of Quantified Statements 9
1.6 Rules of Inference 9
1.7 Mathematical Statements 10
1.8 Basic Proof Methods 10

2 Sets and Functions **11**
2.1 Sets and Basic Operations 12
2.2 Cartesian Products and Power Sets 12
2.3 Functions 12
2.4 Cardinality and Infinite Sets 13
2.5 Floor, Ceiling, and Basic Sequences 13

3 Complexity of Algorithms **14**
3.1 Asymptotic Notation 15
3.2 Time and Space Complexity 15
3.3 Horner's Method 15
3.4 The Class P 15
3.5 The Class NP 16
3.6 NP-Complete and NP-Hard Problems 16

4 Number Theory and Cryptography **17**
4.1 Divisibility 18
4.2 Greatest Common Divisors 18
4.3 Euclidean Algorithm and Bézout's Identity 18
4.4 Prime Numbers 18
4.5 Congruences 19
4.6 Modular Inverses and Linear Congruences 19
4.7 Chinese Remainder Theorem 20
4.8 Fermat's Little Theorem and Euler's Theorem 20
4.9 Primitive Roots 20
4.10 Hash Functions and Pseudorandom Numbers 20
4.11 Classical Cryptography 20
4.12 Discrete Logarithm Problem 21
4.13 RSA Cryptography 21

5 Induction and Recursion **22**
5.1 Mathematical Induction 23
5.2 Strong Induction 23
5.3 Well-Ordering Principle 23
5.4 Recursive Definitions 23
5.5 Recurrence Relations for Algorithms 24

5.6 Master Theorem 24

6 Counting and Generating Functions 25
6.1 Basic Counting Principles 26
6.2 Permutations and Combinations 26
6.3 Pigeonhole Principle 26
6.4 Inclusion–Exclusion Principle 26
6.5 Linear Recurrence Relations 27
6.6 Generating Functions 27
6.7 Standard Generating Functions 27
6.8 Solving Recurrences by Generating Functions 28
6.9 Example of a First-Order Recurrence 28

7 Relations 29
7.1 Binary Relations 30
7.2 Properties of Relations 30
7.3 Matrix Representation of Relations 31
7.4 Composition and Powers of Relations 31
7.5 Closures of Relations 32
7.6 Equivalence Relations 32
7.7 Partitions 32
7.8 Partial Orders 33
7.9 Lexicographic and Well-Orders 33
7.10 Hasse Diagrams 33
7.11 Maximal, Minimal, Greatest, and Least Elements 34
7.12 Upper and Lower Bounds 34

8 Graphs and Trees 36
8.1 Graphs and Basic Terminology 37
8.2 Degree of a Vertex 37
8.3 Subgraphs and Graph Operations 37
8.4 Special Types of Graphs 38
8.5 Bipartite Graphs 38
8.6 Matchings 38
8.7 Graph Representations 38
8.8 Graph Isomorphism 39
8.9 Paths and Connectivity 39
8.10 Counting Paths with Adjacency Matrices 39
8.11 Euler Paths and Circuits 39
8.12 Hamilton Paths and Circuits 40
8.13 Shortest Paths 40
8.14 Planar Graphs 40
8.15 Planarity Criteria 40
8.16 Graph Coloring 41
8.17 Trees 41
8.18 Rooted and m-Ary Trees 41
8.19 Tree Traversal 42
8.20 Spanning Trees 42
8.21 Minimum Spanning Trees 42

Acknowledgements **43**

Introduction

Discrete mathematics studies mathematical structures that are fundamentally discrete rather than continuous. Unlike analysis, where limits, continuity, derivatives, and integrals provide a relatively unified framework, discrete mathematics consists of several closely related but distinct subjects, including logic, sets, algorithms, number theory, combinatorics, relations, graphs, and trees.

Because of this structure, these notes do not attempt to develop the subject mainly through long proofs. Instead, the emphasis is placed on definitions, fundamental theorems, standard notation, useful identities, and the connections among different discrete structures. Proofs are included only when they are short and directly illuminate an important idea.

A recurring theme throughout discrete mathematics is the translation of informal questions into precise mathematical language. Logic formalizes valid reasoning; sets and functions provide a basic language for mathematical objects; asymptotic notation describes the efficiency of algorithms; number theory develops divisibility and modular arithmetic; combinatorics provides systematic counting methods; relations describe structural connections; and graph theory represents pairwise interactions between discrete objects.

The subject also lies close to theoretical computer science. Recurrence relations, generating functions, Boolean matrices, modular arithmetic, graph algorithms, and computational complexity are simultaneously mathematical structures and computational tools.

These notes are organized into eight chapters:

1. Logic and Proofs;
2. Sets and Functions;
3. Complexity of Algorithms;
4. Number Theory and Cryptography;
5. Induction and Recursion;
6. Counting and Generating Functions;
7. Relations;
8. Graphs and Trees.

The goal is to provide a compact reference for the central concepts of an undergraduate course in discrete mathematics, emphasizing definitions, theorems, notation, and structural relationships rather than lengthy derivations.

Chapter 1

Logic and Proofs

1.1 Propositions and Logical Operators

Definition 1.1.1 (Proposition)**.** A *proposition* is a declarative statement with a definite truth value: either true (T) or false (F), but not both.

A proposition obtained by combining one or more propositions using logical operators is called a *compound proposition*.
Let p and q be propositions. The fundamental logical operators are:

Operation	Notation	Meaning
Negation	$\neg p$	not p
Conjunction	$p \wedge q$	p and q
Disjunction	$p \vee q$	p or q
Exclusive or	$p \oplus q$	exactly one of p, q is true
Implication	$p \to q$	if p, then q
Biconditional	$p \leftrightarrow q$	p if and only if q

The corresponding truth table is

p	q	$p \wedge q$	$p \vee q$	$p \oplus q$	$p \to q$	$p \leftrightarrow q$
T	T	T	T	F	T	T
T	F	F	T	T	F	F
F	T	F	T	T	T	F
F	F	F	F	F	T	T

The implication $p \to q$ is false only when p is true and q is false. If p is false, then $p \to q$ is true regardless of the truth value of q.
For the implication $p \to q$:

- $q \to p$ is the *converse*;
- $\neg p \to \neg q$ is the *inverse*;
- $\neg q \to \neg p$ is the *contrapositive*.

The implication and its contrapositive are logically equivalent:

$$p \to q \equiv \neg q \to \neg p.$$

Definition 1.1.2 (Logical Equivalence)**.** Two compound propositions P and Q are *logically equivalent*, written $P \equiv Q$, if they have the same truth value under every possible assignment of truth values to their variables.

Logical equivalence $P \equiv Q$ is a relation between two propositional expressions, whereas $P \leftrightarrow Q$ is itself a proposition.

Definition 1.1.3 (Tautology, Contradiction, and Contingency)**.** A proposition that is always true is called a *tautology*. A proposition that is always false is called a *contradiction*. A proposition that is neither a tautology nor a contradiction is called a *contingency*.

1.2 Logical Equivalences

The following equivalences are frequently used in propositional logic.

Identity laws:

$$p \wedge T \equiv p, \qquad p \vee F \equiv p.$$

Domination laws:

$$p \vee T \equiv T, \qquad p \wedge F \equiv F.$$

Idempotent laws:

$$p \vee p \equiv p, \qquad p \wedge p \equiv p.$$

Double negation law:

$$\neg(\neg p) \equiv p.$$

Commutative laws:

$$p \vee q \equiv q \vee p, \qquad p \wedge q \equiv q \wedge p.$$

Associative laws:

$$(p \vee q) \vee r \equiv p \vee (q \vee r), \qquad (p \wedge q) \wedge r \equiv p \wedge (q \wedge r).$$

Distributive laws:

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r),$$
$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r).$$

De Morgan's laws:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q, \qquad \neg(p \vee q) \equiv \neg p \wedge \neg q.$$

Absorption laws:

$$p \vee (p \wedge q) \equiv p, \qquad p \wedge (p \vee q) \equiv p.$$

Negation laws:

$$p \vee \neg p \equiv T, \qquad p \wedge \neg p \equiv F.$$

Implication can be rewritten as

$$p \to q \equiv \neg p \vee q.$$

Hence

$$\neg(p \to q) \equiv \neg(\neg p \vee q) \equiv p \wedge \neg q.$$

The biconditional satisfies

$$p \leftrightarrow q \equiv (p \to q) \wedge (q \to p).$$

1.3 Predicate Logic

Propositional logic treats complete propositions as indivisible objects. *Predicate logic* extends propositional logic by allowing statements whose truth values depend on variables.

Definition 1.3.1 (Predicate)**.** A *predicate* $P(x)$ is a statement involving a variable x. After a particular value is substituted for x, the expression becomes a proposition.

For example, $P(x) : x^2 > 4$ is a predicate over $\mathbb{R}$. Its truth value depends on the value assigned to x.
The set of allowable values for the variable is called the *domain* or *universe of discourse*. Predicates may involve several variables, such as $P(x, y)$ or $P(x_1, \ldots, x_n)$.

1.4 Quantifiers

The two fundamental quantifiers are the universal quantifier and the existential quantifier.

Definition 1.4.1 (Universal Quantifier)**.** The statement $\forall x\, P(x)$ means that $P(x)$ is true for every x in the specified domain.

Definition 1.4.2 (Existential Quantifier)**.** The statement $\exists x\, P(x)$ means that there exists at least one x in the specified domain for which $P(x)$ is true.

The order of quantifiers is important. In general,

$$\forall x\, \exists y\, P(x, y) \qquad \text{and} \qquad \exists y\, \forall x\, P(x, y)$$

are not equivalent.
For example, over $\mathbb{R}$,

$$\forall x\, \exists y\ (x + y = 0)$$

is true because for every x, one may choose $y = -x$. However,

$$\exists y\, \forall x\ (x + y = 0)$$

is false because no single value of y works for every x.
Quantifiers have higher precedence than the standard logical operators. For nested quantifiers, their order must therefore be read carefully.

1.5 Negation of Quantified Statements

Negating a quantified statement reverses the quantifier and negates the predicate:

$$\neg\forall x\, P(x) \equiv \exists x\, \neg P(x),$$

$$\neg\exists x\, P(x) \equiv \forall x\, \neg P(x).$$

Thus, to disprove a universal statement, it is enough to find a single counterexample.
For nested quantifiers, every quantifier changes when the negation is moved through the statement. For example,

$$\neg\left(\forall x\, \exists y\, P(x, y)\right) \equiv \exists x\, \forall y\, \neg P(x, y).$$

1.6 Rules of Inference

An *argument* consists of propositions called *premises*, followed by a proposition called the *conclusion*. An argument is valid if the truth of all premises guarantees the truth of the conclusion.
Some standard rules of inference are:
Modus Ponens:

$$p, \qquad p \to q \quad \Longrightarrow \quad q.$$

Modus Tollens:

$$\neg q, \qquad p \to q \quad \Longrightarrow \quad \neg p.$$

Hypothetical Syllogism:

$$p \to q, \qquad q \to r \quad \Longrightarrow \quad p \to r.$$

Disjunctive Syllogism:

$$p \vee q, \qquad \neg p \quad \Longrightarrow \quad q.$$

For quantified statements, four standard rules are particularly useful:

- **Universal Instantiation (UI):** from $\forall x\, P(x)$, conclude $P(c)$ for a particular c;
- **Universal Generalization (UG):** if $P(c)$ is established for an arbitrary c, conclude $\forall x\, P(x)$;
- **Existential Instantiation (EI):** from $\exists x\, P(x)$, choose an element c for which $P(c)$ holds;
- **Existential Generalization (EG):** from $P(c)$, conclude $\exists x\, P(x)$.

1.7 Mathematical Statements

Definition 1.7.1 (Axiom)**.** An *axiom* or *postulate* is a statement accepted as true without proof and used as a starting point for a mathematical theory.

Definition 1.7.2 (Theorem)**.** A *theorem* is a mathematical statement established by a valid proof from axioms, definitions, and previously proved results.

Definition 1.7.3 (Lemma)**.** A *lemma* is a theorem used primarily as an intermediate result in the proof of another theorem.

Definition 1.7.4 (Corollary)**.** A *corollary* is a result that follows directly from a theorem or another established result.

Definition 1.7.5 (Conjecture)**.** A *conjecture* is a mathematical statement believed to be true but not yet proved.

1.8 Basic Proof Methods

Although lengthy proofs are not the main focus of these notes, several basic proof methods are part of the standard language of discrete mathematics.
Direct proof. To prove $p \to q$, assume p and derive q.
Proof by contraposition. Instead of proving $p \to q$, prove the logically equivalent statement $\neg q \to \neg p$.
Proof by contradiction. To prove p, assume $\neg p$ and derive a contradiction.
Proof by cases. Separate all possibilities into exhaustive cases and prove the desired statement in each case.
Vacuous proof. An implication $p \to q$ is automatically true if p is false.
Trivial proof. An implication $p \to q$ is automatically true if q is already known to be true.

Chapter 2

Sets and Functions

2.1 Sets and Basic Operations

A *set* is a collection of distinct objects, called its *elements*. If x is an element of a set A, we write $x \in A$; otherwise, $x \notin A$.
For two sets A and B, the fundamental set operations are

$$A \cup B = \{x : x \in A \text{ or } x \in B\},$$
$$A \cap B = \{x : x \in A \text{ and } x \in B\},$$
$$A - B = \{x : x \in A \text{ and } x \notin B\}.$$

If the universe of discourse is U, the complement of A is $A^c = U - A$.
Two sets are equal, written $A = B$, if they contain exactly the same elements.

Definition 2.1.1 (Subset)**.** A set A is a *subset* of B, written $A \subseteq B$, if every element of A also belongs to B.

The cardinality of a finite set A, denoted by $|A|$, is the number of elements in A.
For finite sets,

$$|A \cup B| = |A| + |B| - |A \cap B|\,.$$

2.2 Cartesian Products and Power Sets

Definition 2.2.1 (Cartesian Product)**.** The *Cartesian product* of sets A and B is

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

The order of the components matters, so in general $A \times B \neq B \times A$.
If A and B are finite, then

$$|A \times B| = |A|\,|B|\,.$$

Definition 2.2.2 (Power Set)**.** The *power set* of A, denoted by $\mathcal{P}(A)$, is the set of all subsets of A:

$$\mathcal{P}(A) = \{B : B \subseteq A\}.$$

If $|A| = n$, then

$$|\mathcal{P}(A)| = 2^n.$$

2.3 Functions

Definition 2.3.1 (Function)**.** A function $f : A \to B$ assigns exactly one element $f(a) \in B$ to every element $a \in A$.

The set A is called the *domain*, and B is called the *codomain*. The set of values actually attained,

$$f(A) = \{f(a) : a \in A\},$$

is called the *range* or *image* of f.

Definition 2.3.2 (Injective Function)**.** A function $f : A \to B$ is *injective* or *one-to-one* if

$$f(x) = f(y) \Longrightarrow x = y.$$

Equivalently, $x \neq y \Longrightarrow f(x) \neq f(y)$.

Definition 2.3.3 (Surjective Function)**.** A function $f : A \to B$ is *surjective* or *onto* if every $b \in B$ has at least one preimage:

$$\forall b \in B, \quad \exists a \in A \quad \text{such that} \quad f(a) = b.$$

Equivalently, $f(A) = B$.

Definition 2.3.4 (Bijective Function)**.** A function is *bijective* if it is both injective and surjective. A bijection is also called a *one-to-one correspondence*.

A function $f : A \to B$ has an inverse function $f^{-1} : B \to A$ precisely when f is bijective.

2.4 Cardinality and Infinite Sets

Two sets A and B have the same cardinality if there exists a bijection between them. We then write $|A| = |B|$.
More generally,

$$|A| \leq |B|$$

if there exists an injection from A to B.

Definition 2.4.1 (Countable Set)**.** A set is *countable* if it is finite or has the same cardinality as $\mathbb{N}$.

The integers $\mathbb{Z}$ and rational numbers $\mathbb{Q}$ are countable, whereas the real numbers $\mathbb{R}$ are uncountable.
The power set $\mathcal{P}(\mathbb{N})$ is also uncountable.

2.5 Floor, Ceiling, and Basic Sequences

For $x \in \mathbb{R}$, the *floor* of x, denoted by $\lfloor x \rfloor$, is the greatest integer not exceeding x, while the *ceiling* $\lceil x \rceil$ is the least integer not smaller than x.
An arithmetic progression has the form

$$a,\ a + d,\ a + 2d, \ldots,$$

whereas a geometric progression has the form

$$a,\ ar,\ ar^2, \ldots.$$

For $|r| < 1$, the infinite geometric series satisfies

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1 - r}.$$

Chapter 3

Complexity of Algorithms

3.1 Asymptotic Notation

Asymptotic notation describes the growth rate of a function while ignoring constant factors and lower-order terms.

Definition 3.1.1 (Big-O Notation)**.** We write $f(x) = O(g(x))$ if there exist constants $C > 0$ and k such that

$$|f(x)| \leq C\,|g(x)|$$

whenever $x > k$.

Thus $O(g(x))$ gives an asymptotic upper bound for $f(x)$.

Definition 3.1.2 (Big-Omega Notation)**.** We write $f(x) = \Omega(g(x))$ if there exist constants $C > 0$ and k such that

$$|f(x)| \geq C\,|g(x)|$$

whenever $x > k$.

Thus $\Omega(g(x))$ gives an asymptotic lower bound.

Definition 3.1.3 (Big-Theta Notation)**.** We write $f(x) = \Theta(g(x))$ if

$$f(x) = O(g(x)) \qquad \text{and} \qquad f(x) = \Omega(g(x)).$$

Therefore, $\Theta(g(x))$ describes the asymptotically exact order of growth of $f(x)$.

3.2 Time and Space Complexity

The *time complexity* of an algorithm measures the number of elementary operations required as a function of the input size.
The *space complexity* measures the amount of memory required by the algorithm.
Typical growth rates include

$$O(1), \qquad O(\log n), \qquad O(n), \qquad O(n \log n), \qquad O(n^2), \qquad O(2^n).$$

When n is large, the asymptotic growth rate usually matters more than constant factors.

3.3 Horner's Method

Consider a polynomial

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n.$$

Direct evaluation repeatedly computes powers of x. Horner's method rewrites the polynomial as

$$p(x) = a_0 + x\Big(a_1 + x(a_2 + \cdots + x(a_{n-1} + x a_n)\cdots)\Big).$$

This representation requires only a linear number of additions and multiplications. Hence the time complexity is $O(n)$.

3.4 The Class P

Definition 3.4.1 (Class P)**.** The complexity class P consists of decision problems that can be solved by a deterministic algorithm in polynomial time.

Thus a problem belongs to P if there exists an algorithm whose running time is bounded by $O(n^k)$ for some constant k.

3.5 The Class NP

Definition 3.5.1 (Class NP)**.** The complexity class NP consists of decision problems for which a given candidate solution, or certificate, can be verified in polynomial time.

The definition concerns the efficiency of *verification*, rather than the efficiency of finding the certificate.
Every problem in P is also in NP:

$$P \subseteq NP.$$

Whether

$$P = NP$$

is one of the central open problems in theoretical computer science.

3.6 NP-Complete and NP-Hard Problems

Definition 3.6.1 (NP-Complete)**.** A problem is *NP-complete* if it belongs to NP and is at least as hard as every other problem in NP.

NP-complete problems may therefore be regarded as the hardest problems inside NP.

Definition 3.6.2 (NP-Hard)**.** A problem is *NP-hard* if it is at least as hard as every problem in NP.

An NP-hard problem does not necessarily belong to NP. Thus NP-completeness imposes the additional requirement that the problem itself be in NP.
If a polynomial-time algorithm were found for any NP-complete problem, then every problem in NP would have a polynomial-time algorithm.

Chapter 4

Number Theory and Cryptography

4.1 Divisibility

For integers a and b, we say that a *divides* b, written $a \mid b$, if there exists an integer k such that $b = ak$.
Some fundamental divisibility properties are

$$a \mid b \Longrightarrow a \mid (bm + an)$$

for all integers m, n,

$$a \mid b, \quad b \mid c \Longrightarrow a \mid c,$$

and

$$a \mid b \Longrightarrow a \mid bc.$$

The division algorithm states that for integers a and positive d, there exist unique integers q and r such that

$$a = dq + r, \qquad 0 \leq r < d.$$

In computational notation,

$$q = a \operatorname{div} d, \qquad r = a \bmod d.$$

4.2 Greatest Common Divisors

Definition 4.2.1 (Greatest Common Divisor)**.** The greatest common divisor of integers a and b, not both zero, is

$$\gcd(a, b) = \max\{d : d \mid a \text{ and } d \mid b\}.$$

Two integers are *coprime* or *relatively prime* if $\gcd(a, b) = 1$.
The least common multiple is denoted by $\operatorname{lcm}(a, b)$.

4.3 Euclidean Algorithm and Bézout's Identity

Lemma 4.3.1. If

$$a = bq + r,$$

then

$$\gcd(a, b) = \gcd(b, r).$$

Repeated application of this identity gives the *Euclidean algorithm* for computing greatest common divisors.

Theorem 4.3.2 (Bézout's Identity)**.** For integers a and b, there exist integers s and t such that

$$\gcd(a, b) = sa + tb.$$

The coefficients s and t can be found by reversing the steps of the Euclidean algorithm.

4.4 Prime Numbers

Definition 4.4.1 (Prime Number)**.** An integer $p > 1$ is *prime* if its only positive divisors are 1 and p.

Two classical families and conjectures appearing in number theory are Mersenne primes, primes of the form $2^p - 1$, and the Goldbach and twin prime conjectures.

Theorem 4.4.2 (Fundamental Theorem of Arithmetic)**.** Every integer $n > 1$ can be written uniquely, apart from the order of the factors, as

$$n = p_1^{\alpha_1} \cdots p_k^{\alpha_k},$$

where $p_1, \ldots, p_k$ are distinct primes and $\alpha_1, \ldots, \alpha_k$ are positive integers.

To test whether n is prime, it is sufficient to check divisibility by primes not exceeding $\sqrt{n}$.

4.5 Congruences

Definition 4.5.1 (Congruence)**.** For integers a, b and $m > 0$,

$$a \equiv b \pmod{m}$$

if $m \mid (a - b)$.

Equivalently,

$$a \equiv b \pmod{m} \iff a \bmod m = b \bmod m.$$

Congruences are compatible with addition, subtraction, and multiplication. For example,

$$a \equiv b \pmod{m} \implies a + c \equiv b + c \pmod{m}$$

and

$$a \equiv b \pmod{m} \implies ac \equiv bc \pmod{m}.$$

Cancellation requires additional care. If

$$ac \equiv bc \pmod{m}$$

and $\gcd(c, m) = 1$, then

$$a \equiv b \pmod{m}.$$

4.6 Modular Inverses and Linear Congruences

An integer a has a multiplicative inverse modulo m if there exists x such that

$$ax \equiv 1 \pmod{m}.$$

Theorem 4.6.1. The integer a has an inverse modulo m if and only if

$$\gcd(a, m) = 1.$$

Such an inverse can be found using Bézout's identity and the Euclidean algorithm. Consider the linear congruence

$$ax \equiv b \pmod{m}.$$

Let $d = \gcd(a, m)$. A solution exists precisely when $d \mid b$. In that case, there are d incongruent solutions modulo m.

4.7 Chinese Remainder Theorem

Theorem 4.7.1 (Chinese Remainder Theorem)**.** Suppose $m_1, \ldots, m_n$ are pairwise coprime positive integers. Then the system

$$x \equiv a_i \pmod{m_i}, \qquad i = 1, \ldots, n,$$

has a unique solution modulo

$$M = m_1 m_2 \cdots m_n.$$

4.8 Fermat's Little Theorem and Euler's Theorem

Theorem 4.8.1 (Fermat's Little Theorem)**.** If p is prime and $\gcd(a, p) = 1$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

The Euler totient function $\varphi(n)$ counts the integers in $\{1, \ldots, n\}$ that are relatively prime to n.

Theorem 4.8.2 (Euler's Theorem)**.** If $\gcd(a, n) = 1$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

For distinct primes p and q,

$$\varphi(pq) = (p-1)(q-1).$$

4.9 Primitive Roots

For a prime p, an integer g is called a *primitive root modulo* p if its powers generate all nonzero residue classes modulo p:

$$\{g, g^2, \ldots, g^{p-1}\} \pmod{p} = \{1, 2, \ldots, p-1\}.$$

Primitive roots are closely connected with the multiplicative structure of integers modulo a prime and with the discrete logarithm problem.

4.10 Hash Functions and Pseudorandom Numbers

A *hash function* maps data of arbitrary size to values in a fixed range and is frequently used to represent or identify data compactly.
A *pseudorandom number generator* produces a deterministic sequence designed to imitate random behavior.
A standard example is the linear congruential generator

$$X_{n+1} = (aX_n + c) \bmod m,$$

where a, c, and m are fixed parameters.

4.11 Classical Cryptography

Cryptography studies methods for transforming information so that only authorized parties can recover the original message.
In a shift cipher, letters are represented modulo 26. Encryption has the form

$$C \equiv M + k \pmod{26},$$

while decryption is

$$M \equiv C - k \pmod{26}.$$

A substitution cipher replaces each symbol by another symbol according to a permutation of the alphabet. For an alphabet with 26 symbols, there are 26! possible substitutions. In a *one-time pad*, the key is a random binary string of the same length as the message and is used only once.

4.12 Discrete Logarithm Problem

Given integers b and y, the *discrete logarithm problem* asks for an exponent x satisfying

$$b^x \equiv y \pmod{m}.$$

A common setting is modulo a large prime p, with b chosen as a primitive root. The difficulty of recovering x from b and y forms the basis of several cryptographic constructions.

4.13 RSA Cryptography

RSA is based on arithmetic modulo a product of two large primes.
Choose distinct large primes p and q, and define

$$n = pq, \qquad \varphi(n) = (p-1)(q-1).$$

Choose an encryption exponent e satisfying

$$\gcd(e, \varphi(n)) = 1.$$

Then choose d such that

$$ed \equiv 1 \pmod{\varphi(n)}.$$

The public encryption key is (n, e), while d is kept private.
A message m is encrypted by

$$c \equiv m^e \pmod{n}.$$

The ciphertext is decrypted by

$$m \equiv c^d \pmod{n}.$$

Since $ed \equiv 1 \pmod{\varphi(n)}$, the exponents e and d are chosen so that modular exponentiation recovers the original message.
RSA can also be used for digital signatures. A signature is generated by

$$s \equiv m^d \pmod{n},$$

and verified using

$$s^e \equiv m \pmod{n}.$$

Chapter 5

Induction and Recursion

5.1 Mathematical Induction

Mathematical induction is a standard method for proving statements indexed by the positive integers.

Theorem 5.1.1 (Principle of Mathematical Induction)**.** Let $P(n)$ be a proposition defined for integers $n \geq n_0$. Suppose that

1. $P(n_0)$ is true;
2. for every $k \geq n_0$, $P(k) \to P(k+1)$.

Then $P(n)$ is true for every integer $n \geq n_0$.

The first step is called the *basis step*. The assumption that $P(k)$ is true is called the *inductive hypothesis*, and proving $P(k+1)$ from it is called the *inductive step*.
The logical structure is

$$P(n_0), \qquad P(k) \to P(k+1) \quad \Longrightarrow \quad P(n)$$

for all $n \geq n_0$.

5.2 Strong Induction

In ordinary induction, the inductive step assumes only $P(k)$. In *strong induction*, all preceding statements may be assumed.

Theorem 5.2.1 (Strong Induction)**.** Let $P(n)$ be defined for integers $n \geq n_0$. Suppose $P(n_0)$ is true and, for every $k \geq n_0$,

$$P(n_0) \wedge P(n_0+1) \wedge \cdots \wedge P(k) \Longrightarrow P(k+1).$$

Then $P(n)$ is true for every $n \geq n_0$.

Strong induction is particularly useful when the case $P(k+1)$ depends on several earlier cases rather than only on $P(k)$.
Ordinary induction and strong induction are logically equivalent as proof principles, although one form may be substantially more convenient for a particular problem.

5.3 Well-Ordering Principle

Theorem 5.3.1 (Well-Ordering Principle)**.** Every nonempty subset of the positive integers has a least element.

The well-ordering principle, ordinary induction, and strong induction are closely related formulations of the same basic property of the integers.
A typical well-ordering argument assumes that a desired statement fails, considers the set of counterexamples, and chooses its least element. The minimality of this counterexample is then used to obtain a contradiction.

5.4 Recursive Definitions

A sequence can be defined *recursively* by specifying initial values and a rule that determines later terms from earlier ones.
For example, a first-order recurrence relation may have the form

$$a_n = F(a_{n-1}),$$

while a recurrence of order k may have the form

$$a_n = F(a_{n-1}, a_{n-2}, \ldots, a_{n-k}).$$

To determine a sequence uniquely, enough initial conditions must be supplied to start the recurrence.
Recursive definitions also occur for functions, sets, algorithms, and combinatorial structures.

5.5 Recurrence Relations for Algorithms

Running times of recursive algorithms are frequently described by recurrence relations.
A common form is

$$T(n) = aT\left(\frac{n}{b}\right) + cn^d,$$

where $a \geq 1$, $b > 1$, $c > 0$, and $d \geq 0$.
Repeated substitution gives

$$T(n) = a^k T\left(\frac{n}{b^k}\right) + cn^d \sum_{j=0}^{k-1} \left(\frac{a}{b^d}\right)^j.$$

When $n = b^k$, the recursion terminates after $k = \log_b n$ levels.

5.6 Master Theorem

Theorem 5.6.1 (Master Theorem)**.** Consider

$$T(n) = aT\left(\frac{n}{b}\right) + cn^d,$$

where $a \geq 1$, $b > 1$, $c > 0$, and $d \geq 0$. Then the asymptotic behavior of $T(n)$ is determined by comparing a with b^d.

1. If $a < b^d$, then
$$T(n) = \Theta(n^d).$$
2. If $a = b^d$, then
$$T(n) = \Theta(n^d \log n).$$
3. If $a > b^d$, then
$$T(n) = \Theta\left(n^{\log_b a}\right).$$

The theorem expresses the competition between the recursive contribution $aT(n/b)$ and the nonrecursive work cn^d.
For example, if

$$T(n) = 2T\left(\frac{n}{2}\right) + n,$$

then $a = 2$, $b = 2$, and $d = 1$, so $a = b^d$. Therefore

$$T(n) = \Theta(n \log n).$$

Chapter 6

Counting and Generating Functions

6.1 Basic Counting Principles

The basic rule of product states that if one procedure can be performed in m ways and, independently, a second procedure can be performed in n ways, then the two procedures can be performed successively in mn ways.
The rule of sum states that if two mutually exclusive alternatives can occur in m and n ways, respectively, then there are $m + n$ possibilities in total.
For $n \geq 1$, the factorial is

$$n! = n(n-1)\cdots 2 \cdot 1,$$

with $0! = 1$.

6.2 Permutations and Combinations

The number of ways to arrange r distinct objects selected from n distinct objects is

$$P(n,r) = \frac{n!}{(n-r)!}.$$

The number of ways to select r objects from n distinct objects when order does not matter is the binomial coefficient

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}.$$

The symmetry relation

$$\binom{n}{r} = \binom{n}{n-r}$$

follows immediately from the formula.
The binomial theorem states that

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

6.3 Pigeonhole Principle

Theorem 6.3.1 (Pigeonhole Principle)**.** If more than n objects are placed into n boxes, then at least one box contains at least two objects.

The generalized form states that if N objects are distributed among k boxes, then at least one box contains at least

$$\left\lceil \frac{N}{k} \right\rceil$$

objects.
The principle is simple but useful for proving existence statements without explicitly identifying the required object.

6.4 Inclusion–Exclusion Principle

For two finite sets,

$$|A \cup B| = |A| + |B| - |A \cap B|\,.$$

For three finite sets,

$$\begin{aligned} |A \cup B \cup C| = {} & |A| + |B| + |C| \\ & - |A \cap B| - |A \cap C| - |B \cap C| \\ & + |A \cap B \cap C|\,. \end{aligned}$$

More generally,

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} \left|\bigcap_{i=1}^{n} A_i\right|.$$

The signs alternate because elements belonging to several sets are successively overcounted and undercounted.

6.5 Linear Recurrence Relations

A linear homogeneous recurrence relation of order k with constant coefficients has the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

To solve the recurrence, consider a trial solution $a_n = r^n$. Substitution gives the characteristic equation

$$r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_k = 0.$$

If the characteristic equation has distinct roots $r_1, \ldots, r_k$, then

$$a_n = \alpha_1 r_1^n + \cdots + \alpha_k r_k^n,$$

where the constants α_i are determined by the initial conditions.
If a root r has multiplicity m, its contribution has the form

$$\left(\alpha_0 + \alpha_1 n + \cdots + \alpha_{m-1} n^{m-1}\right) r^n.$$

6.6 Generating Functions

Definition 6.6.1 (Generating Function)**.** For a sequence $\{a_n\}_{n\geq 0}$, its ordinary generating function is

$$G(x) = a_0 + a_1 x + a_2 x^2 + \cdots = \sum_{n=0}^{\infty} a_n x^n.$$

The sequence is encoded as the coefficients of a formal power series. Generating functions convert recurrence relations and counting problems into algebraic problems involving power series.

6.7 Standard Generating Functions

Several generating functions occur repeatedly:

$$\frac{1}{1-x} = \sum_{n=0}^{\infty} x^n,$$

$$\frac{1}{1-ax} = \sum_{n=0}^{\infty} a^n x^n,$$

$$\frac{1}{(1-x)^2} = \sum_{n=0}^{\infty} (n+1) x^n,$$

and more generally,

$$\frac{1}{(1-x)^m} = \sum_{n=0}^{\infty} \binom{n+m-1}{m-1} x^n.$$

The binomial expansions

$$(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$$

and

$$(1-x)^{-m} = \sum_{k=0}^{\infty} \binom{m+k-1}{k} x^k$$

are particularly useful in counting problems.
The exponential series is

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

6.8 Solving Recurrences by Generating Functions

Consider the linear recurrence

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

Let

$$G(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Multiplying the recurrence by x^n and summing over n produces an equation involving

$$G(x), \quad xG(x), \quad x^2G(x), \ldots, \quad x^kG(x).$$

After collecting terms, the generating function has the general form

$$G(x) = \frac{P(x)}{1 - c_1 x - c_2 x^2 - \cdots - c_k x^k},$$

where the polynomial $P(x)$ is determined by the initial conditions.
The denominator can then be factored and expanded, often by partial fractions, to recover an explicit expression for a_n.

6.9 Example of a First-Order Recurrence

Consider

$$a_n = 5a_{n-1} - 6, \qquad a_0 = 3.$$

Let

$$G(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Using the recurrence,

$$G(x) - a_0 = 5xG(x) - 6\sum_{n=1}^{\infty} x^n.$$

Since

$$\sum_{n=1}^{\infty} x^n = \frac{x}{1-x},$$

we obtain

$$G(x) - 3 = 5xG(x) - \frac{6x}{1-x}.$$

Hence

$$G(x) = \frac{3 - 9x}{(1-x)(1-5x)}.$$

After partial fraction decomposition, the coefficients of the resulting power series determine the closed form of a_n.

Chapter 7

Relations

7.1 Binary Relations

Definition 7.1.1 (Binary Relation)**.** A *binary relation* R from a set A to a set B is a subset of the Cartesian product $A \times B$:

$$R \subseteq A \times B.$$

If $(a, b) \in R$, we write aRb.

A binary relation *on* a set A is a relation from A to itself, so $R \subseteq A \times A$.
If $|A| = m$ and $|B| = n$, then $A \times B$ contains mn ordered pairs. Since every relation from A to B is a subset of $A \times B$, the number of possible binary relations is

$$2^{mn}.$$

In particular, the number of binary relations on an n-element set is 2^{n^2}.

7.2 Properties of Relations

Let R be a binary relation on a set A.

Definition 7.2.1 (Reflexive Relation)**.** The relation R is *reflexive* if

$$(a, a) \in R$$

for every $a \in A$.

Definition 7.2.2 (Irreflexive Relation)**.** The relation R is *irreflexive* if

$$(a, a) \notin R$$

for every $a \in A$.

Definition 7.2.3 (Symmetric Relation)**.** The relation R is *symmetric* if

$$(a, b) \in R \Longrightarrow (b, a) \in R.$$

Definition 7.2.4 (Antisymmetric Relation)**.** The relation R is *antisymmetric* if

$$(a, b) \in R \quad \text{and} \quad (b, a) \in R \Longrightarrow a = b.$$

Equivalently, if $a \neq b$, then aRb and bRa cannot both hold.
Notice that *antisymmetric* does not mean "not symmetric." A relation may be both symmetric and antisymmetric.

Definition 7.2.5 (Transitive Relation)**.** The relation R is *transitive* if

$$(a, b) \in R \quad \text{and} \quad (b, c) \in R \Longrightarrow (a, c) \in R.$$

7.3 Matrix Representation of Relations

Suppose

$$A = \{a_1, \ldots, a_m\}, \qquad B = \{b_1, \ldots, b_n\}.$$

A relation R from A to B can be represented by an $m \times n$ Boolean matrix $M_R = (m_{ij})$, where

$$m_{ij} = \begin{cases} 1, & a_i R b_j, \\ 0, & a_i \not R b_j. \end{cases}$$

For a relation on $A = \{a_1, \ldots, a_n\}$, its properties can be read from the matrix:

- R is reflexive if every diagonal entry satisfies $m_{ii} = 1$;
- R is irreflexive if every diagonal entry satisfies $m_{ii} = 0$;
- R is symmetric if $M_R = M_R^T$;
- R is antisymmetric if $m_{ij} = 1$ and $i \neq j$ imply $m_{ji} = 0$.

Set operations on relations correspond to Boolean operations on their matrices. For example,

$$M_{R \cup S} = M_R \vee M_S, \qquad M_{R \cap S} = M_R \wedge M_S,$$

where the operations are performed entrywise.

7.4 Composition and Powers of Relations

Let R be a relation from A to B, and let S be a relation from B to C.

Definition 7.4.1 (Composition of Relations)**.** The composition $S \circ R$ is the relation from A to C defined by

$$a(S \circ R)c$$

if there exists $b \in B$ such that aRb and bSc.

In terms of Boolean matrices,

$$M_{S \circ R} = M_R \odot M_S,$$

where $\odot$ denotes Boolean matrix multiplication.
For a relation R on A, define

$$R^1 = R, \qquad R^{n+1} = R^n \circ R.$$

Then

$$(a, b) \in R^n$$

means that there exists a path of length n from a to b using the relation R.
A relation R is transitive precisely when

$$R^2 \subseteq R.$$

7.5 Closures of Relations

A *closure* of a relation is obtained by adding the minimum required ordered pairs so that the resulting relation satisfies a desired property.
The *reflexive closure* of R on A is

$$R \cup \{(a, a) : a \in A\}.$$

The *symmetric closure* is

$$R \cup R^{-1},$$

where

$$R^{-1} = \{(b, a) : (a, b) \in R\}.$$

The *transitive closure* of R contains (a, b) whenever there is a path of positive length from a to b.
It can be expressed as

$$R^+ = R \cup R^2 \cup R^3 \cup \cdots .$$

If paths of length zero are also allowed, the corresponding connectivity relation is

$$R^* = I \cup R \cup R^2 \cup R^3 \cup \cdots ,$$

where $I = \{(a, a) : a \in A\}$ is the identity relation.
For a finite relation represented by an $n \times n$ Boolean matrix, the transitive closure can be computed by a path algorithm in $O(n^3)$ time.

7.6 Equivalence Relations

Definition 7.6.1 (Equivalence Relation)**.** A relation R on a set S is an *equivalence relation* if it is reflexive, symmetric, and transitive.

Equivalence relations formalize the idea that two elements should be regarded as belonging to the same class.

Definition 7.6.2 (Equivalence Class)**.** If R is an equivalence relation on S, the *equivalence class* of $a \in S$ is

$$[a]_R = \{b \in S : (a, b) \in R\}.$$

For an equivalence relation,

$$aRb \iff [a]_R = [b]_R.$$

Furthermore, any two equivalence classes are either identical or disjoint:

$$[a]_R \cap [b]_R \neq \varnothing \implies [a]_R = [b]_R.$$

Thus equivalence classes divide the original set into nonoverlapping pieces.

7.7 Partitions

Definition 7.7.1 (Partition)**.** A *partition* of a set S is a collection of nonempty subsets $\{A_i\}_{i \in I}$ such that

$$S = \bigcup_{i \in I} A_i$$

and

$$A_i \cap A_j = \varnothing \qquad \text{whenever } i \neq j.$$

Equivalence relations and partitions describe exactly the same structure.

Theorem 7.7.2. Every equivalence relation on S determines a partition of S into equivalence classes. Conversely, every partition of S determines an equivalence relation whose equivalence classes are precisely the sets in the partition.

7.8 Partial Orders

Definition 7.8.1 (Partial Order)**.** A relation R on a set S is a *partial order* if it is reflexive, antisymmetric, and transitive.

A set equipped with a partial order is called a *partially ordered set*, or *poset*, and is written (S, R), often as $(S, \preceq)$.
Two elements $a, b \in S$ are *comparable* if either $a \preceq b$ or $b \preceq a$. Otherwise, they are *incomparable*.

Definition 7.8.2 (Total Order)**.** A partial order is a *total order* if every pair of elements is comparable.

Thus every total order is a partial order, but a partial order need not be total.
The usual ordering $\leq$ on $\mathbb{R}$ is a total order, whereas divisibility on the positive integers is only a partial order.

7.9 Lexicographic and Well-Orders

The *lexicographic order* compares ordered tuples by examining the first coordinate in which they differ. It generalizes the usual dictionary ordering of words.

Definition 7.9.1 (Well-Ordered Set)**.** A totally ordered set is *well-ordered* if every nonempty subset has a least element.

The positive integers with their usual order are well-ordered. This property is the basis of the well-ordering principle and provides another formulation of mathematical induction.

7.10 Hasse Diagrams

Finite posets can be represented compactly by *Hasse diagrams*.
Starting from the directed graph of a partial order:

1. remove all loops implied by reflexivity;
2. remove all edges implied by transitivity;
3. place larger elements above smaller elements;
4. omit arrowheads, since upward direction represents the order.

An edge from a upward to b means that b *covers* a: $a \prec b$ and there is no c satisfying

$$a \prec c \prec b.$$

The complete partial order can be recovered from the Hasse diagram using reflexivity and transitivity.

7.11 Maximal, Minimal, Greatest, and Least Elements

Let A be a subset of a poset $(S, \preceq)$.
An element $a \in A$ is *maximal* in A if there is no $b \in A$ such that

$$a \prec b.$$

Similarly, $a \in A$ is *minimal* if there is no $b \in A$ such that

$$b \prec a.$$

An element $g \in A$ is the *greatest element* of A if

$$a \preceq g$$

for every $a \in A$.
An element $l \in A$ is the *least element* of A if

$$l \preceq a$$

for every $a \in A$.
A poset may have several maximal or minimal elements. However, if a greatest or least element exists, it is unique.
Therefore,

$$\text{greatest} \implies \text{maximal},$$

but a maximal element need not be greatest. Similarly,

$$\text{least} \implies \text{minimal},$$

but a minimal element need not be least.

7.12 Upper and Lower Bounds

Let $A \subseteq S$, where $(S, \preceq)$ is a poset.
An element $u \in S$ is an *upper bound* of A if

$$a \preceq u$$

for every $a \in A$.
An element $l \in S$ is a *lower bound* of A if

$$l \preceq a$$

for every $a \in A$.
An upper or lower bound does *not* have to belong to A.

Definition 7.12.1 (Least Upper Bound)**.** A *least upper bound* of A, also called the *supremum*, is an upper bound u such that

$$u \preceq v$$

for every upper bound v of A.

Definition 7.12.2 (Greatest Lower Bound)**.** A *greatest lower bound* of A, also called the *infimum*, is a lower bound l such that

$$v \preceq l$$

for every lower bound v of A.

When they exist, the least upper bound and greatest lower bound are unique.
The distinction between these concepts is important:

Concept	Must belong to A?
maximal/minimal element	Yes
greatest/least element	Yes
upper/lower bound	No
least upper/greatest lower bound	No

Chapter 8

Graphs and Trees

8.1 Graphs and Basic Terminology

Definition 8.1.1 (Graph)**.** A *graph* is an ordered pair

$$G = (V, E),$$

where V is the set of *vertices* and E is the set of *edges*.

For an undirected edge connecting vertices u and v, the vertices u and v are called its *endpoints*. Two vertices are *adjacent* or *neighbors* if they are joined by an edge.

Definition 8.1.2 (Simple Graph)**.** A *simple graph* is an undirected graph with no loops and no multiple edges between the same pair of vertices.

A *pseudograph* may contain loops and multiple edges.
In a directed graph, or *digraph*, an edge is an ordered pair (u, v). The vertex u is the *initial vertex*, and v is the *terminal vertex*.
The neighborhood of a vertex v is

$$N(v) = \{u \in V : \{u, v\} \in E\}.$$

For $A \subseteq V$, define

$$N(A) = \bigcup_{v \in A} N(v).$$

8.2 Degree of a Vertex

The *degree* of a vertex v, denoted by $\deg(v)$, is the number of edges incident with v, with a loop counted twice.
A vertex with degree 0 is called *isolated*, while a vertex with degree 1 is called *pendant*.

Theorem 8.2.1 (Handshaking Theorem)**.** For every finite undirected graph $G = (V, E)$,

$$\sum_{v \in V} \deg(v) = 2\,|E|\,.$$

Corollary 8.2.2. Every finite undirected graph contains an even number of vertices of odd degree.

For a directed graph, the *in-degree* $\deg^-(v)$ counts edges entering v, while the *out-degree* $\deg^+(v)$ counts edges leaving v. Then

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|\,.$$

8.3 Subgraphs and Graph Operations

Definition 8.3.1 (Subgraph)**.** A graph $H = (W, F)$ is a *subgraph* of $G = (V, E)$ if

$$W \subseteq V \qquad \text{and} \qquad F \subseteq E.$$

If $H \neq G$, then H is a *proper subgraph*.
For $W \subseteq V$, the *induced subgraph* on W contains every edge of G whose two endpoints belong to W.
Deleting an edge e gives

$$G - e = (V, E - \{e\}),$$

while adding an edge gives

$$G + e = (V, E \cup \{e\}).$$

Deleting a vertex v removes v together with all edges incident with v.

8.4 Special Types of Graphs

The *complete graph* K_n is the simple graph in which every pair of distinct vertices is adjacent. It has

$$|E(K_n)| = \binom{n}{2} = \frac{n(n-1)}{2}.$$

The *cycle graph* C_n consists of n vertices arranged in one cycle.
The *wheel graph* W_n is obtained from a cycle by adding one central vertex adjacent to every vertex of the cycle.
The n-dimensional *hypercube* Q_n has binary strings of length n as vertices. Two vertices are adjacent precisely when their binary representations differ in exactly one position.

8.5 Bipartite Graphs

Definition 8.5.1 (Bipartite Graph)**.** A graph $G = (V, E)$ is *bipartite* if

$$V = V_1 \cup V_2, \qquad V_1 \cap V_2 = \varnothing,$$

and every edge has one endpoint in V_1 and the other in V_2.

Thus there are no edges joining two vertices within the same part.

Theorem 8.5.2. A graph is bipartite if and only if its vertices can be colored with two colors so that adjacent vertices receive different colors.

Equivalently, a graph is bipartite if and only if it contains no odd cycle.
The *complete bipartite graph* $K_{m,n}$ has parts of sizes m and n, with every vertex in one part adjacent to every vertex in the other. Therefore,

$$|E(K_{m,n})| = mn.$$

8.6 Matchings

Definition 8.6.1 (Matching)**.** A *matching* is a set of edges no two of which share a common vertex.

A *maximum matching* is a matching containing the largest possible number of edges.
A matching from V_1 to V_2 is *complete* if every vertex in V_1 is matched to a distinct vertex in V_2.

Theorem 8.6.2 (Hall's Marriage Theorem)**.** Let $G = (V_1 \cup V_2, E)$ be a bipartite graph. There exists a matching that matches every vertex of V_1 if and only if

$$|N(A)| \geq |A|$$

for every subset $A \subseteq V_1$.

8.7 Graph Representations

A graph may be represented using an *adjacency list*, which records the neighbors of each vertex.
For a graph with vertices $v_1, \ldots, v_n$, its *adjacency matrix* is the matrix $A = (a_{ij})$, where a_{ij} records the number of edges from v_i to v_j.
For a simple graph,

$$a_{ij} = \begin{cases} 1, & v_i \text{ and } v_j \text{ are adjacent,} \\ 0, & \text{otherwise.} \end{cases}$$

For an undirected simple graph, the adjacency matrix is symmetric.
The *incidence matrix* records the incidence relation between vertices and edges.

8.8 Graph Isomorphism

Definition 8.8.1 (Graph Isomorphism)**.** Graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there exists a bijection

$$f : V_1 \to V_2$$

such that

$$\{u, v\} \in E_1 \quad \Longleftrightarrow \quad \{f(u), f(v)\} \in E_2.$$

Isomorphic graphs have the same graph structure even if their vertices have different labels or geometric positions.
Consequently, isomorphic graphs have the same number of vertices, edges, and vertices of each degree. These conditions are useful for showing that two graphs are not isomorphic, although they are not sufficient in general to prove isomorphism.

8.9 Paths and Connectivity

A *path* is a sequence of vertices

$$v_0, v_1, \ldots, v_k$$

such that consecutive vertices are adjacent. Its length is k, the number of edges used.
A *simple path* does not repeat vertices.
A *circuit* is a path that begins and ends at the same vertex.

Definition 8.9.1 (Connected Graph)**.** An undirected graph is *connected* if there exists a path between every pair of vertices.

A *connected component* is a maximal connected subgraph.
An edge whose removal increases the number of connected components is called a *bridge* or *cut edge*.
For a directed graph, *strong connectivity* means that for every $u, v \in V$, there is a directed path from u to v and a directed path from v to u.

8.10 Counting Paths with Adjacency Matrices

Let A be the adjacency matrix of a graph.

Theorem 8.10.1. The (i, j)-entry of A^r equals the number of walks of length r from vertex v_i to vertex v_j.

Thus matrix multiplication provides an algebraic method for counting walks in a graph.

8.11 Euler Paths and Circuits

Definition 8.11.1 (Euler Path)**.** An *Euler path* is a path that contains every edge of a graph exactly once.

An *Euler circuit* is an Euler path that begins and ends at the same vertex.

Theorem 8.11.2 (Euler Circuit Criterion)**.** A connected undirected graph has an Euler circuit if and only if every vertex has even degree.

Theorem 8.11.3 (Euler Path Criterion)**.** A connected undirected graph has an Euler path but no Euler circuit if and only if exactly two vertices have odd degree.

Therefore:

Number of odd-degree vertices	Conclusion
0	Euler circuit exists
2	Euler path exists
> 2	No Euler path

8.12 Hamilton Paths and Circuits

Definition 8.12.1 (Hamilton Path)**.** A *Hamilton path* is a simple path that visits every vertex exactly once.

A *Hamilton circuit* is a Hamilton path whose final vertex is adjacent to its initial vertex, forming a cycle through all vertices.
Euler problems concern using every *edge*, whereas Hamilton problems concern visiting every *vertex*.
Unlike Euler paths, there is no simple necessary-and-sufficient criterion based only on vertex degrees for determining whether a general graph has a Hamilton path or circuit.
The Hamilton circuit problem is closely related to NP-complete problems.

8.13 Shortest Paths

A *weighted graph* assigns a numerical weight to each edge. The length of a path is the sum of the weights of its edges.
The shortest-path problem asks for a path of minimum total weight between two vertices.

Theorem 8.13.1 (Dijkstra's Algorithm)**.** For a graph with nonnegative edge weights, Dijkstra's algorithm determines shortest-path distances from a chosen source vertex to all reachable vertices.

A basic implementation has complexity $O(n^2)$, while implementations using suitable priority queues can be substantially faster for sparse graphs.

8.14 Planar Graphs

Definition 8.14.1 (Planar Graph)**.** A graph is *planar* if it can be drawn in the plane without any edges crossing except at common endpoints.

A planar drawing divides the plane into *regions*, including the unbounded exterior region. If $\deg(R)$ denotes the number of edge occurrences on the boundary of a region R, then

$$\sum_{R} \deg(R) = 2\,|E|\,.$$

Theorem 8.14.2 (Euler's Formula)**.** For a connected planar graph with V vertices, E edges, and R regions,

$$V - E + R = 2.$$

Equivalently,

$$R = E - V + 2.$$

8.15 Planarity Criteria

An *elementary subdivision* replaces an edge $\{u, v\}$ by two edges $\{u, w\}$ and $\{w, v\}$, where w is a new vertex.
Two graphs are *homeomorphic* if they can be obtained from the same graph by sequences of elementary subdivisions.

Theorem 8.15.1 (Kuratowski's Theorem)**.** A graph is nonplanar if and only if it contains a subgraph homeomorphic to K_5 or $K_{3,3}$.

Thus K_5 and $K_{3,3}$ are the two fundamental obstructions to planarity.

8.16 Graph Coloring

A *proper vertex coloring* assigns colors to vertices so that adjacent vertices receive different colors.

Definition 8.16.1 (Chromatic Number)**.** The *chromatic number* $\chi(G)$ is the minimum number of colors required for a proper coloring of G.

Some standard examples are

$$\chi(K_n) = n, \qquad \chi(K_{m,n}) = 2$$

for $m, n \geq 1$.
For a cycle C_n,

$$\chi(C_n) = \begin{cases} 2, & n \text{ even}, \\ 3, & n \text{ odd}. \end{cases}$$

Theorem 8.16.2 (Four Color Theorem)**.** Every simple planar graph can be properly colored using at most four colors:

$$\chi(G) \leq 4.$$

8.17 Trees

Definition 8.17.1 (Tree)**.** A *tree* is a connected undirected graph containing no simple circuits.

Trees admit several equivalent characterizations.

Theorem 8.17.2. For an undirected graph $T = (V, E)$, the following are equivalent:

1. T is a tree;
2. T is connected and contains no simple circuits;
3. there exists a unique simple path between every pair of vertices;
4. T is connected and $|E| = |V| - 1$.

Thus, if a tree has n vertices, then it has exactly $n - 1$ edges.

8.18 Rooted and m-Ary Trees

A *rooted tree* is a tree in which one vertex is distinguished as the *root*.
For a non-root vertex, its neighboring vertex immediately closer to the root is its *parent*. Vertices directly below a vertex are its *children*. A vertex with no children is a *leaf*; otherwise it is an *internal vertex*.
A *subtree rooted at* v consists of v together with all of its descendants.

Definition 8.18.1 (m-Ary Tree)**.** A rooted tree is an m-ary tree if every internal vertex has at most m children.

It is a *full m-ary tree* if every internal vertex has exactly m children.
If a full m-ary tree has i internal vertices, then

$$n = mi + 1$$

vertices in total, and the number of leaves is

$$l = (m - 1)i + 1.$$

An m-ary tree of height h has at most

$$m^h$$

leaves.

8.19 Tree Traversal

Tree traversal specifies an order in which the vertices of a rooted tree are visited.
For an ordered rooted tree, *preorder traversal* visits the root first and then recursively traverses the subtrees from left to right:

$$\text{root} \to \text{left} \to \text{right}.$$

For a binary tree, *inorder traversal* follows

$$\text{left} \to \text{root} \to \text{right}.$$

Postorder traversal recursively traverses the subtrees before visiting the root:

$$\text{left} \to \text{right} \to \text{root}.$$

For binary expression trees, these traversal orders correspond naturally to prefix, infix, and postfix representations.

8.20 Spanning Trees

Definition 8.20.1 (Spanning Tree)**.** A *spanning tree* of a connected graph G is a subgraph that contains every vertex of G and is itself a tree.

Theorem 8.20.2. Every connected graph has a spanning tree.

A spanning tree can be constructed using either depth-first search or breadth-first search.
Depth-first search (DFS) explores as far as possible along one branch before backtracking.
Breadth-first search (BFS) explores vertices level by level, visiting all nearby vertices before moving farther from the starting vertex.
Using adjacency lists, both DFS and BFS can be implemented in $O(|V| + |E|)$ time.

8.21 Minimum Spanning Trees

For a connected weighted graph, a *minimum spanning tree* is a spanning tree whose total edge weight is as small as possible.
Two standard algorithms are Prim's algorithm and Kruskal's algorithm.

Prim's algorithm begins with a single vertex and repeatedly adds a minimum-weight edge connecting the current tree to a new vertex.
Kruskal's algorithm considers edges in increasing order of weight and repeatedly adds an edge whenever doing so does not create a cycle.
With appropriate data structures, both algorithms can achieve running times on the order of

$$O(|E| \log |V|)$$

or equivalently $O(|E| \log |E|)$ up to the usual graph-size relations.
Prim's algorithm is often convenient for dense graphs, while Kruskal's algorithm is particularly natural for sparse graphs.

Acknowledgements

I would like to express my sincere gratitude to Professor Shan Chen for the lecture slides used in this course.
Discrete mathematics contains many topics that may initially appear separate from one another. Professor Shan Chen's course materials helped organize these individual concepts into a coherent network, making it much easier to understand their connections and to see how ideas from logic, counting, relations, graph theory, and other parts of the subject fit together.
This structure was especially helpful when reviewing and reorganizing the course into these notes. I am grateful for the clarity of the presentation and for the way the lectures connected otherwise scattered pieces of discrete mathematics into a more unified picture.